Rischi e checklist

1. Rischi principali

Rischio	Come si riconosce	Prevenzione	Rimedio
Posizioni dei token sbagliate	la testa non impara; F1 vicino a E-RAND	test di 03 §5 (decodifica dei token di fine frase)	correggere la mappa degli offset (escaping JSON di Winnow)
Segmentazione diversa fra training e produzione	buono in valutazione, frasi sfasate in produzione	un solo modulo `sentences.py`, con hash in `decision.json`	riaddestrare la testa se il segmentatore cambia
Evidenza sparsa non trovata	F1 alto su FEVER, basso su HoVer e oltre 10 frasi	aggregatore bidirezionale, claim prima, righe sparse costruite	più dati sparsi; H3c (più layer); fase 2
Verdetto peggiorato	BAcc o ECE del verdetto cambiano	fase 1 con backbone congelato; λ basso in fase 2	tornare alla fase 1; ancoraggio β·KL
Annotazioni incomplete	frasi giuste contate come errori; precisione bassa ma sufficienza alta	ranking, ω basso per FEVER; sufficienza come metrica	leggere a mano i "falsi positivi"
Bias di posizione	evidenzia spesso la prima frase o il titolo	righe sparse con evidenza in posizioni varie; H9	togliere l'embedding di posizione
Scorciatoia lessicale	la testa ≈ E-LEX; fallisce su parafrasi e frasi ponte	dati con parafrasi (C2D, D2C), frasi ponte (HoVer, CG2C)	più dati di questo tipo
Etichette per ablazione rumorose	H8: aggiungerle peggiora	giudice LLM, peso 0,5	escluderle
Documenti oltre il contesto	troncamento silenzioso	`max_document_tokens` in `decision.json`; segnalazione	spezzare a monte, sapendo che l'evidenza fra pezzi si perde
Server che non espone gli stati	impossibile servire con vLLM/llama.cpp	verificare presto (settimana 1)	server proprio con transformers
Latenza	p95 sopra +3%	trasferire solo `q` e `h_k`; aggregatore piccolo	ridurre layer dell'aggregatore
Frasi lunghe	evidenziato un paragrafo intero	spezzare frasi > 60 parole	granularità più fine (fuori perimetro)
Fughe	distrattori o etichette da righe di test	generazione e ablazione solo sul train	rifare gli split di evidenza

2. Checklist per step

Step 0 — Split

- ☐ Sottoinsiemi `dev_evid_*`, `test_evid_id`, `test_evid_ood`, `test_evid_sparse` congelati.
- ☐ Distribuzione della distanza fra frasi gold registrata.

Step 0a — Approccio

- ☐ Architettura approvata (claim prima, stati di fine frase, aggregatore).
- ☐ Budget di latenza e lunghezza massima del documento fissati.

Step 1 — Baseline

- ☐ B-ORD misurato.
- ☐ E-LEX, E-EMB, E-ATT su `dev_evid_select`; E-ABL su campione.

Step 2a — Fonti

- ☐ FEVER dalle `lines`; HoVer riallineato (100 pagine verificate); SciFact e WiCE come test.
- ☐ `sentences` coerenti con `document` per tutte le righe.

Step 2b — Etichette generate

- ☐ C2D con frasi marcate; CG2C con indice di frase per tripla.
- ☐ Ablazione offline con conferma del giudice $\geq 80\%$.
- ☐ Righe sparse senza riferimenti rotti (pronomi).

Step 2c — Verdetto

- ☐ Scelta fase 1 / fase 2; probabilità del verdetto precalcolate se serve l'ancoraggio.

Step 2d — Formato

- ☐ `question.json` con `state_order = [claim, document]`.

Step 2e — Mix

- ☐ `ev_head` (e `ev_joint`) con $\geq 40\%$ di righe a più frasi di evidenza e $\geq 25\%$ oltre 10 frasi.

Step 3 — Rendering

- ☐ Test di posizione, escaping e coerenza training/produzione superati.

Step 4 — Loss

- ☐ Funzioni testate sugli esempi numerici; α e ω fissati.

Step 5 — Training

- ☐ Stati estratti verificati; overfit di 64 righe.
- ☐ `τ_e` su `dev_evid_select`, confermata.
- ☐ (Fase 2) verdetto entro i limiti.

Step 6 — Valutazione

- ☐ F1, insieme completo, fasce di distanza, sufficienza.
- ☐ Verdetto identico (fase 1) o entro i limiti (fase 2).
- ☐ Latenza $\leq +3\%$.

Step 7 — Esperimenti

- ☐ H0–H9 registrati; controlli C1–C3.

Step 8 — Export

- ☐ Server che restituisce lettere, `q` e `h_k` da un forward.
- ☐ Offset sul testo originale verificati.
- ☐ `frozen.json` prima del test; model card con "evidenzia solo se supportato".